

```

int checkPositive(int x) {
    if (x > 0) {
        return 1;
    }
    return 0;
}

int sumRange(int n) {
    int sum = 0;
    int i = 0;
    while (i < n) {
        sum = sum + i;
        i = i + 1;
    }
    return sum;
}

int main() {
    int result = checkPositive(5);
    int total = sumRange(10);
    return 0;
}

```

The parser generates an Abstract Syntax Tree where the root is a Program node containing three function definitions. Each function has a FunctionDecl node with parameters, return type, and a body containing statement nodes. For checkPositive, the body includes an IfStmt node with a condition ($x > 0$), a then-branch (ReturnStmt with value 1), and an implicit else-branch (another ReturnStmt with value 0). For sumRange, the body contains variable declarations for sum and i, followed by a WhileStmt node containing a condition ($i < n$) and a compound statement body with assignment and increment operations. The main function contains two variable declarations with initialization expressions that are CallExpr nodes referencing the other functions.

The symbol table uses a hierarchical scope structure. The global scope contains entries for all three functions: checkPositive (type: $\text{int} \rightarrow \text{int}$), sumRange (type: $\text{int} \rightarrow \text{int}$), and main (type: $\text{void} \rightarrow \text{int}$). When entering checkPositive, a new scope is created containing parameter x (type: int, offset: 0). For sumRange, its scope contains parameter n plus local variables sum and i (all type: int, with stack offsets 0, -4, -8 respectively). The main function scope contains result and total (type: int, offsets -4, -8). Scope management follows a stack discipline: when the parser enters a function or block, it pushes a new

scope onto the scope stack; when exiting, it pops the scope, ensuring variables are only visible within their defining context and preventing name collisions between different function scopes.